

Tower Guard Sentinel

Architecture Flowchart

Telecom mast security platform for Nigeria — Bun + Turborepo monorepo



Contents

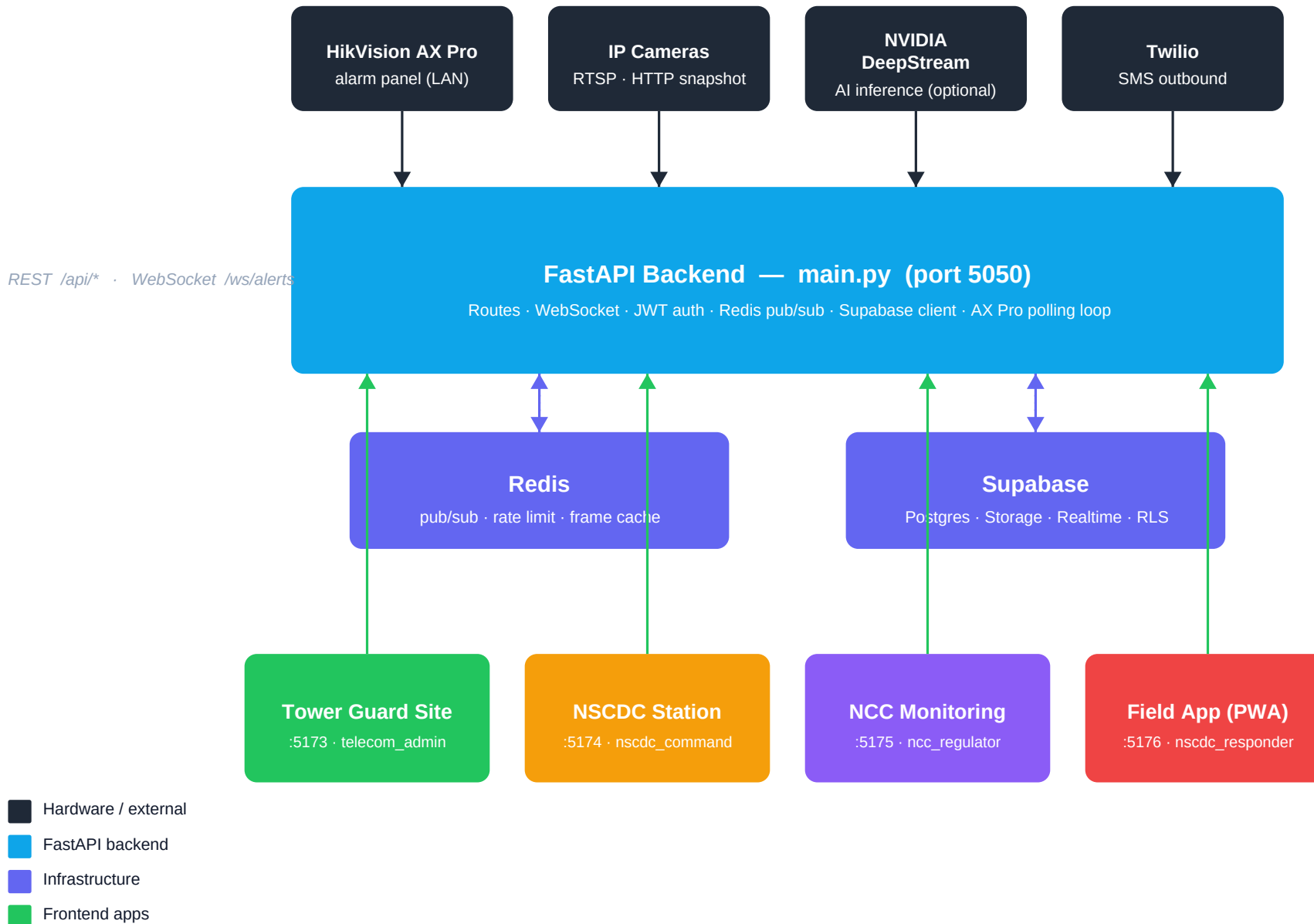
1. System Architecture — hardware, backend, frontend
2. AX Pro Sensor Data Flow — panel to UI
3. Dispatch Flow — Tower Guard to NSCDC to Field
4. Authentication & Authorization — 3 layers of defense
5. Deployment Topology — dev and production

Pair this document with *README.md* and *NAMING_CONVENTIONS.md* for a complete handover.

© Seismic Consulting Group — Internal

1. System Architecture

Four React apps share one FastAPI backend, one Supabase database, and hardware integrations.



2. AX Pro Sensor Data Flow

How a zone trigger on the alarm panel reaches a user's screen.

Known bugs in this flow

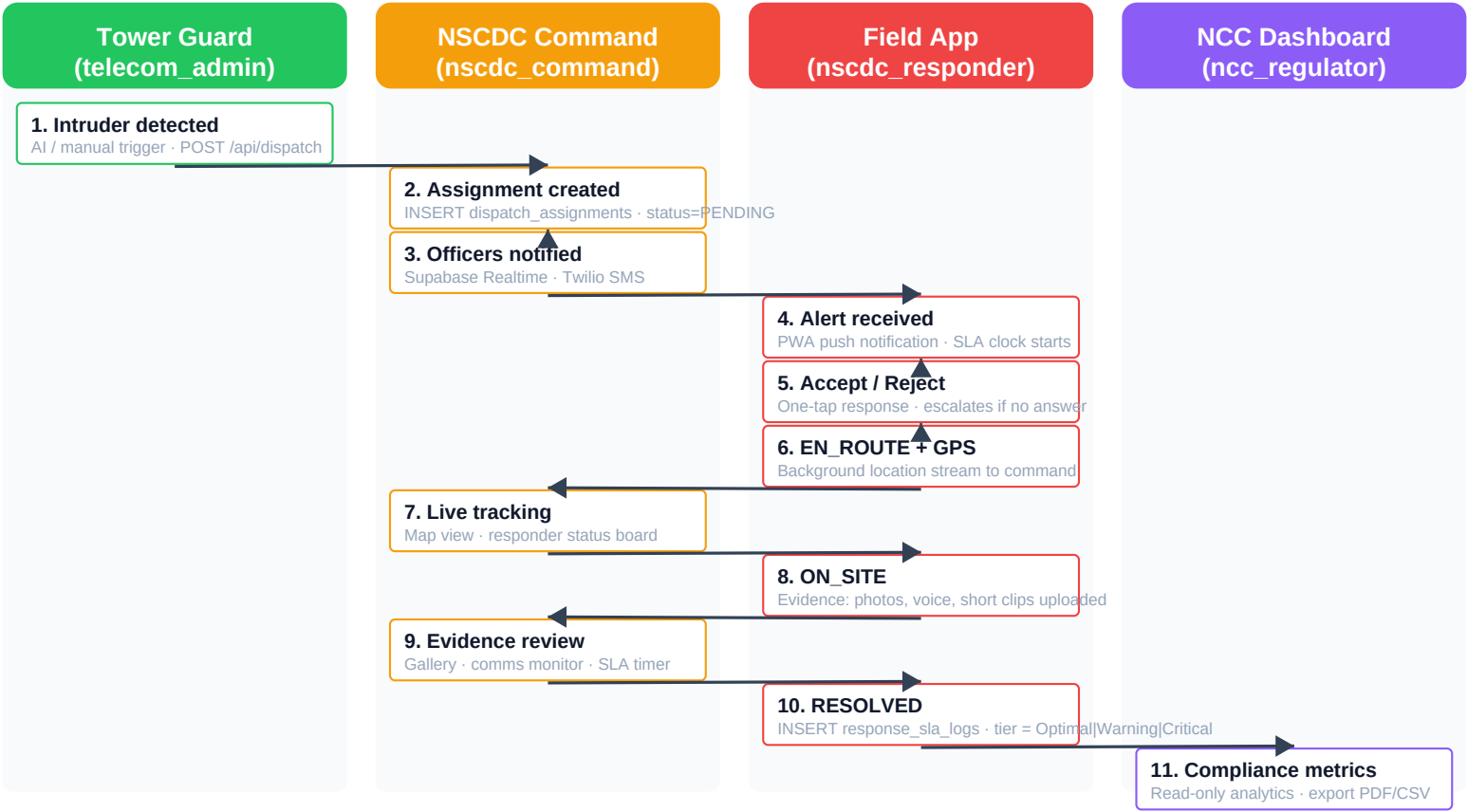
see *README §16* for fixes

- BUG-1** stale axpro_client reference in routes.py
- BUG-4** redis_subscriber never restarts after Redis drop
- BUG-5** polling loop calls have no timeout
- BUG-6** siren_alarm produced but unhandled in useSimulation
- BUG-7** non-connection errors log-spam forever
- BUG-9** active_websockets list has no lock



3. Dispatch Flow — Uber-style

Tower Guard requests · NSCDC Command coordinates · Field App responds · NCC observes.

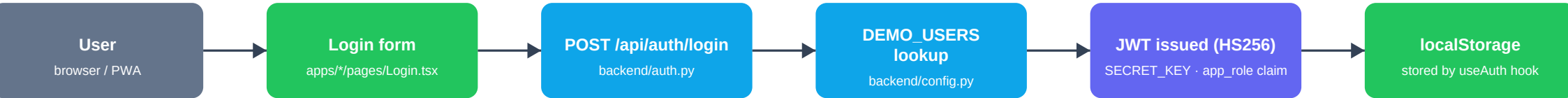


Uber analogy

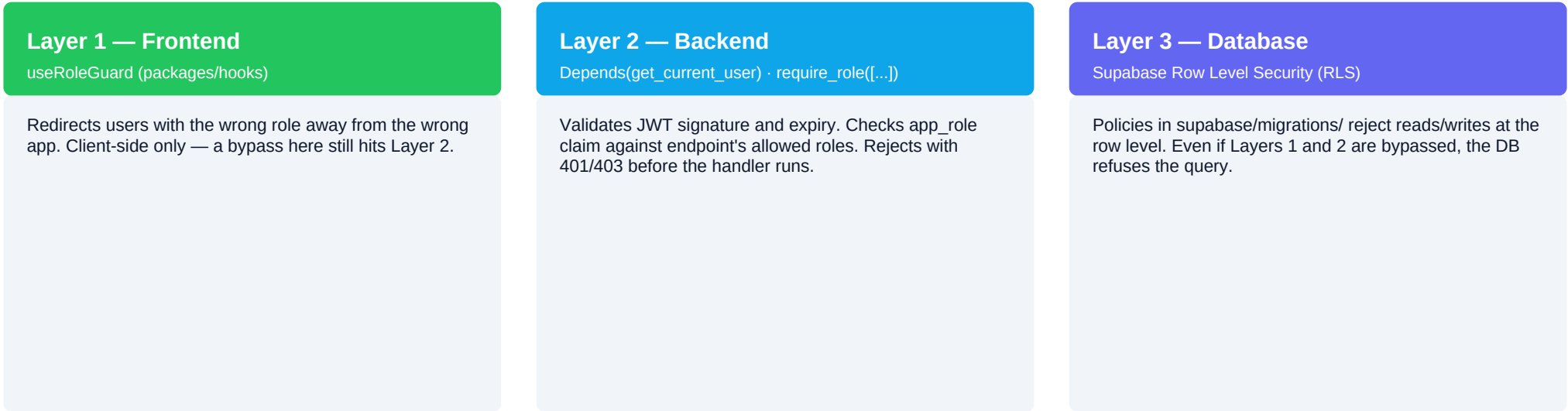
Tower Guard = rider · Field App = driver · NSCDC Station = Uber HQ · NCC = transport regulator
SLA tiers: Optimal (<10m) · Warning (10–20m) · Critical (>20m)

4. Authentication & Authorization

Three layers of defense · all three must pass.



Defense in depth — every request passes through three gates



Roles (packages/data/src/types/app-role.ts)

ROLE	DASHBOARD	SCOPE
telecom_admin	Tower Guard Site Dashboard	Full CRUD on masts, alerts, dispatch
nscdc_command	NSCDC Station Dashboard	Manage assignments · view comms
nscdc_responder	Field App PWA	Accept/reject · upload evidence
ncc_regulator	NCC Monitoring Dashboard	Read-only analytics · export reports

Generated from backend/axpro.py, backend/routes.py, backend/websocket.py, packages/hooks/src/useSimulation.ts

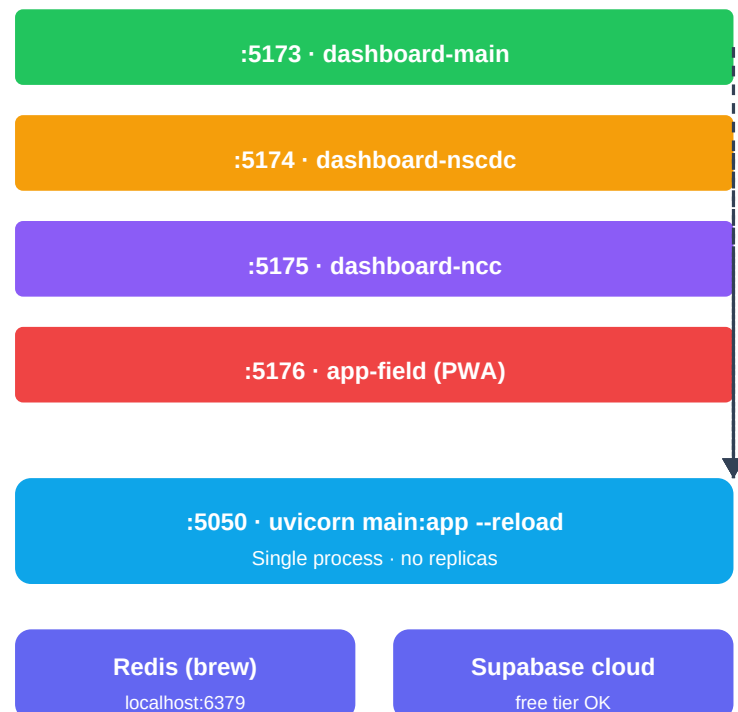
See README §10 for demo logins and JWT lifecycle.

5. Deployment Topology

Dev and production layouts. Each frontend app ships as its own artifact.

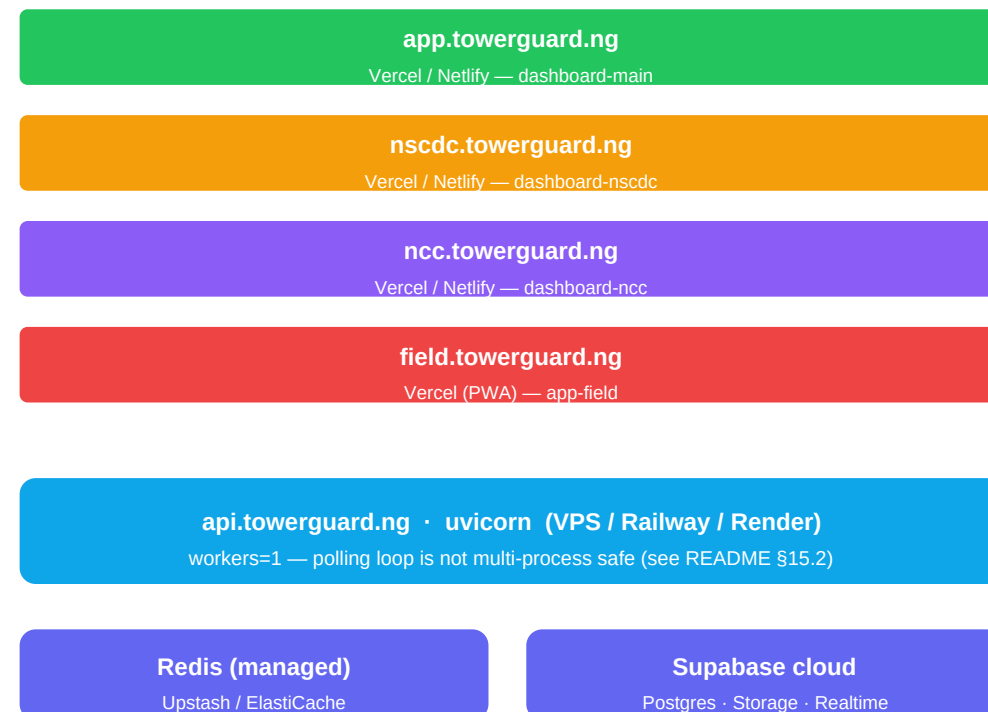
Development (localhost)

Single machine · Redis via Homebrew · Supabase cloud



Production

Each app on its own subdomain · backend + Redis on VPS · Supabase cloud



Edge / on-site equipment (per mast)

HikVision AX Pro alarm panel · IP cameras (RTSP) · Vibration / PIR / door sensors · Optional NVIDIA Jetson for DeepStream AI
LAN → VPN → backend. Polling loop runs server-side; edge devices do not need public internet.
Generator fuel · battery charge · solar output are surfaced as energy telemetry when wired.